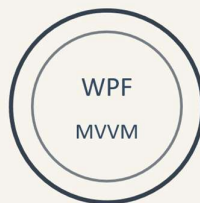


2026

MVVM in der Praxis mit WPF

Architektur, Umsetzung und Teststrategie
am Projekt „Taschenrechner“



Dipl.-Inf. Kamal Mohamed

Technische Dokumentation • .NET 8 • MVVM • Unit-Tests

Inhaltsverzeichnis

Kapitel 1: Einleitung	3
Kapitel 2: Überblick über das MVVM-Pattern	4
Kapitel 3 : Beschreibung der Komponenten	5
Kapitel 3.2: View – Darstellung in XAML.....	9
Kapitel 3.3 : ViewModel – Bindbare Properties und Anwendungslogik	11
Kapitel 4 : Datenbindung (Binding) im Projekt	13
Kapitel 5 : Verwendung von Commands im Projekt.....	14
Kapitel 6 : Kommunikation zwischen Komponenten.....	17
Kapitel 7 : Projektstruktur (Ordner, Dateien, Verantwortlichkeiten)	18
Kapitel 8 : Vorteile und mögliche Nachteile von MVVM im Kontext dieses Projekts	20
Kapitel 8.1 : Vorteile	20
Kapitel 8.2 : Mögliche Nachteile / Verbesserungsfelder.....	20
Kapitel 9 : Fazit	21
Kapitel 10: Model im MVVM – CalculatorModel im Projekt „Taschenrechner“	21
1. Aufgaben und Abgrenzung des Models	22
2. Konkretes Codebeispiel aus dem Projekt	22
3. Erklärung des Codes im MVVM-Kontext.....	24
4. Warum dieses Model MVVM-konform ist.....	26
5. Direkter Bezug zum restlichen Projekt (kurze Einordnung)	26
Kapitel 11 : Unit-Tests im MVVM-Projekt – CalculatorModelTests als Qualitätsnachweis für das Model.....	27
1. Einordnung – Was wird hier getestet und warum?	27
2. Konkretes Codebeispiel aus dem Projekt.....	28
3. Erklärung der Teststruktur (Arrange – Act – Assert)	29
4. Setup und Test-Isolation	29
5. Tests für Exceptions – fachliche Regeln als definierte Erwartungen.....	30
6. Floating-Point-Tests mit Delta (Sin, Log10)	31
Kurzes Zwischenfazit: Warum diese Tests für MVVM besonders wertvoll sind	31
Kapitel 12 : Testabdeckung erweitern und ViewModel-Tests einordnen (Projekt „Taschenrechner“).....	32
1. Sinnvolle Ergänzungen für CalculatorModelTests (Model-Ebene)	32
2. ViewModel-Tests im MVVM-Kontext – was sinnvoll ist (und was nicht)	37
Anhang : Taschenrechner herunterladen und mit Visual Studio Code starten (Windows – einfach erklärt)	39
1. Was du brauchst (Windows).....	39
2. Projekt herunterladen	40

3. Visual Studio Code (kostenlos) installieren.....	41
4. .NET 8 SDK installieren (damit das Programm starten kann)	42
5. Projekt in VS Code öffnen	43
6. Programm starten (Windows)	44
7. Häufige Fehler (kurz) und was du dann machst	45

Kapitel 1: Einleitung

Dieses Buch dokumentiert ein bestehendes WPF-Projekt „**Taschenrechner**“ (C#, .NET 8, WPF) und erklärt daran das **MVVM (Model-View-ViewModel)**

Pattern praxisnah.

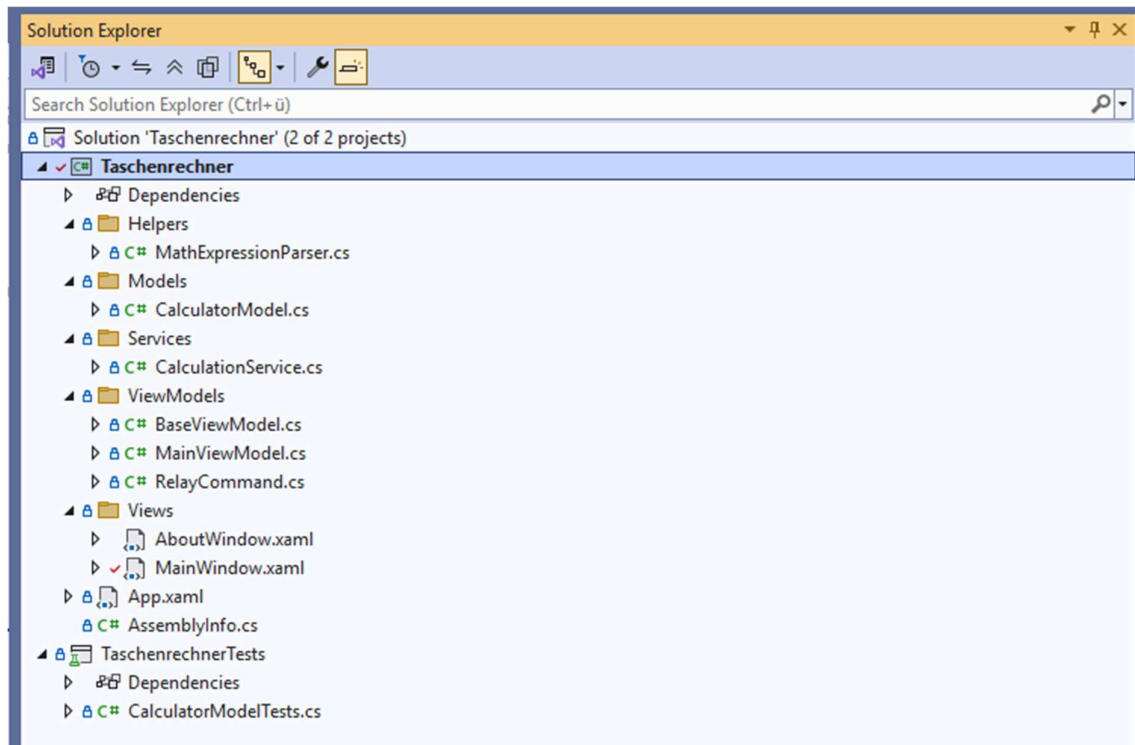
Der Fokus liegt nicht auf einer theoretischen Abhandlung, sondern auf der konkreten Umsetzung im Quellcode:

- Welche Klassen übernehmen welche Rolle
- Wie funktioniert **Binding**
- Wie werden **Commands** eingesetzt
- Wie kommunizieren die Komponenten miteinander.

Das Projekt ist bewusst überschaubar gehalten- Es gibt:

- Eine Hauptansicht (MainWindow)
- Ein zentrales ViewModel (MainViewModel)
- Ein fachliches Model (CalculatorModel), ergänzt durch einen Service (CalculationService)
- Einen Helper (MathExpressionParser).

Gerade diese klare Struktur eignet sich gut, um MVVM anhand realer Stellen im Code zu erklären.



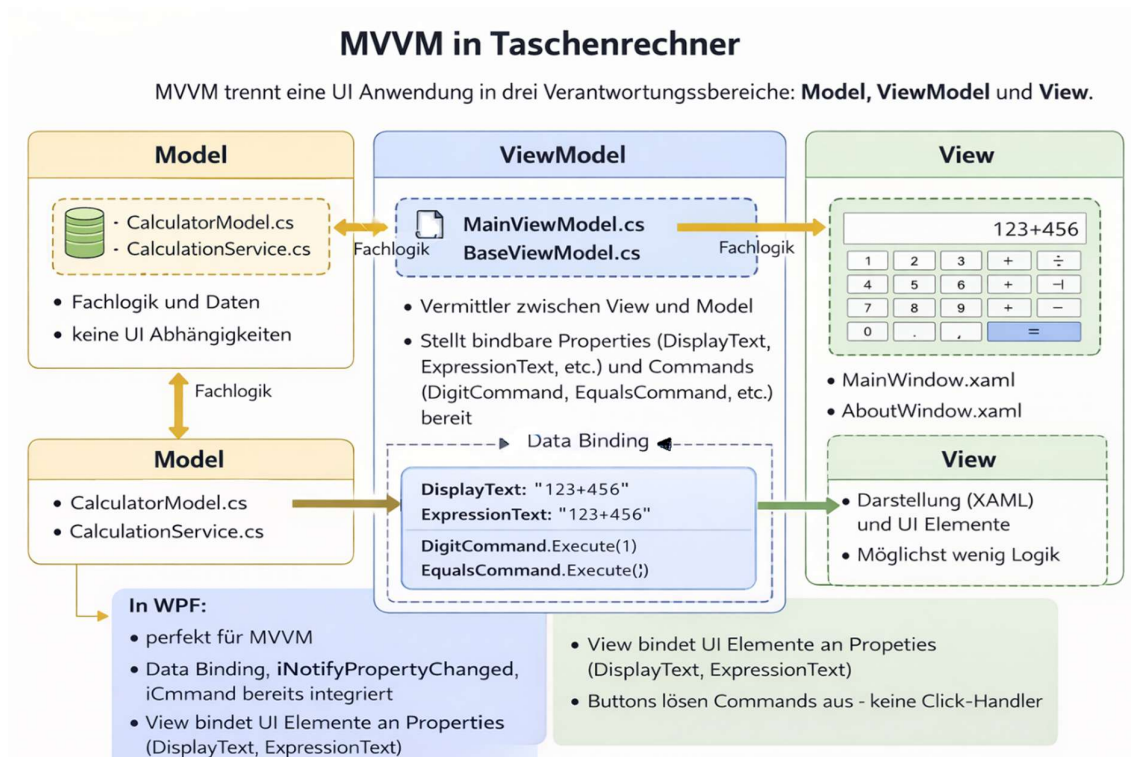
Kapitel 2: Überblick über das MVVM-Pattern

MVVM trennt eine UI-Anwendung in drei Verantwortungsbereiche:

- **Model:** Fachlogik und Daten; keine UI-Abhängigkeiten.
- **View:** Darstellung (XAML) und UI-Elemente; möglichst wenig Logik.
- **ViewModel:** Vermittler zwischen View und Model; stellt bindbare Properties und Commands bereit.

In WPF spielt MVVM seine Stärken aus, weil WPF mit **Data Binding**, **INotifyPropertyChanged** und **ICommand** bereits die passenden technischen Bausteine mitbringt. Im Projekt „Taschenrechner“ sieht man diese Grundidee sehr direkt:

- Die View bindet UI-Elemente an Properties wie **DisplayText** und **ExpressionText**.
- Buttons lösen keine Click-Handler aus, sondern rufen **Commands** wie **DigitCommand** oder **EqualsCommand** auf.
- Die fachliche Berechnung liegt in **CalculatorModel** und **CalculationService**.



Kapitel 3 : Beschreibung der Komponenten

Kapitel 3.1 : Model – Fachlogik und Regeln

Im Projekt ist das zentrale Model die Klasse **CalculatorModel**. Sie kapselt mathematische Operationen und validiert Eingaben (z. B. Division durch 0).

Code-Beispiel (Model):

../Taschenrechner/Models/CalculatorModel.cs

```

public class CalculatorModel
{
    // Grundlegende Operationen
    public double Add(double a, double b) => a + b;
    public double Subtract(double a, double b) => a - b;
    public double Multiply(double a, double b) => a * b;

    public double Divide(double a, double b)
    {
        if (b == 0)
            throw new DivideByZeroException("Division durch Null ist nicht möglich");
        return a / b;
    }

    // Erweiterte Operationen
    public double Power(double baseNum, double exponent) => Math.Pow(baseNum, exponent);

    public double SquareRoot(double value)
    {
        if (value < 0)
            throw new ArgumentException("Keine reelle Wurzel für negative Zahlen");
        return Math.Sqrt(value);
    }

    // Trigonometrische Funktionen (im Bogenmaß)
    public double Sin(double angle) => Math.Sin(angle);
    public double Cos(double angle) => Math.Cos(angle);

    public double Tan(double angle)
    {
        if (Math.Cos(angle) == 0)
            throw new ArgumentException("Tangens ist an diesem Punkt nicht definiert");
        return Math.Tan(angle);
    }

    // Weitere nützliche Funktionen
    public double Factorial(double n)
    {
        if (n < 0 || n != Math.Floor(n))
            throw new ArgumentException("Fakultät ist nur für nicht-negative Ganzzahlen definiert");

        if (n == 0 || n == 1)
            return 1;

        double result = 1;
        for (int i = 2; i <= n; i++)
            result *= i;

        return result;
    }

    public double Percent(double value) => value / 100;
}

```

Einordnung im MVVM-Kontext:

- **Wichtig (fett):** Das Model enthält **keine** WPF-Typen, kein Binding, keine Commands.
- Fehler-fälle werden fachlich sauber über Exceptions beschrieben (z. B. **DivideByZeroException**). Das ViewModel entscheidet anschließend, wie die UI darauf reagiert (z. B. Anzeige „Fehler“).

Ergänzend existiert fachliche Logik außerhalb des Models:

- **CalculationService** (Service-Schicht): wertet zusammengesetzte Ausdrücke aus.
- **MathExpressionParser** (Helper): demonstriert eine alternative Auswertung/Ersetzung spezieller Funktionen.

Beispiel **CalculationService**:

Code-Beispiel (Service):

```
public class CalculationService
{
    private readonly CalculatorModel _calculator;

    public CalculationService()
    {
        _calculator = new CalculatorModel();
    }

    public double EvaluateExpression(string expression)
    {
        expression = ReplaceAdvancedFunctions(expression);

        try
        {
            object result = new DataTable().Compute(expression, null);
            return Convert.ToDouble(result);
        }
        catch (Exception ex)
        {
            throw new ArgumentException($"Fehler bei der Auswertung des Au
        }
    }

    private string ReplaceAdvancedFunctions(string expression)
    {
        return expression
            .Replace("sin(", "Math.Sin(")
            .Replace("cos(", "Math.Cos(")
            .Replace("tan(", "Math.Tan(")
            .Replace("sqrt(", "Math.Sqrt(")
            .Replace("log(", "Math.Log10(")
            .Replace("ln(", "Math.Log(");
    }
}
```

Einordnung:

- Der Service ist fachlich orientiert, aber nicht UI-abhängig. In MVVM ist das typisch: komplexere Use-Cases werden gerne in Services ausgelagert, damit das ViewModel nicht zu einem „God Object“ wird.

Kapitel 3.2: View – Darstellung in XAML

Die View ist im Projekt **MainWindow.xaml**. Hier wird kein Rechenalgorithmus implementiert, sondern die Oberfläche beschrieben: Textanzeigen, Grid-Layout, Buttons, Styles – und vor allem **Binding** und **Command-Zuweisungen**.

Code-Beispiel (View – DataContext):

.../Taschenrechner/Views/MainWindow.xaml

```
<Window.DataContext>
    <viewModel:MainViewModel />
</Window.DataContext>
```

Einordnung:

- Die View instanziert das ViewModel direkt. Das ist für kleine Projekte praktikabel. In größeren Anwendungen würde man typischerweise Dependency Injection oder einen ViewModel-Locator einsetzen.

Die Textausgabe erfolgt per Binding:

Code-Beispiel (View – Binding):

.../Taschenrechner/Views/MainWindow.xaml

```
<TextBlock Text="{Binding ExpressionText}" ... />
<TextBlock Grid.Column="0" Text="{Binding AngleModeText}" ... />
<TextBlock Grid.Column="1" Text="{Binding DisplayText}" ... />
```

Buttons sind konsequent an Commands gebunden:

Code-Beispiel (View – Commands):

.../Taschenrechner/Views/MainWindow.xaml

```
<Button Content="7"
        Command="{Binding DigitCommand}"
        CommandParameter="7"
        Style="{StaticResource DigitButtonStyle}" />
```

Einordnung:

- Die View kennt nur den Command-Namen (**DigitCommand**) und liefert ein **CommandParameter** („7“).
- Welche Aktion daraus folgt, entscheidet ausschließlich das ViewModel.

Hinweis zur Code-Behind-Datei:

Die Datei **MainWindow.xaml.cs** enthält dennoch Logik, nämlich Keyboard-Handling. Das ist gezielt lösungsorientiertes Handeln: Tastatureingaben werden im Code-Behind abgefangen und dann an das ViewModel delegiert.

Code-Beispiel (View Code-Behind – Delegation an VM):

.../Taschenrechner/Views/MainWindow.xaml.cs

```
private void Window_KeyDown(object sender, KeyEventArgs e)
{
    if (e.Key >= Key.D0 && e.Key <= Key.D9)
    {
        string digit = (e.Key - Key.D0).ToString();
        (DataContext as MainViewModel)?.OnKeyInput(digit);
    }
    ...
    else if (e.Key == Key.Enter || e.Key == Key.Return)
    {
        (DataContext as MainViewModel)?.OnKeyInput("=");
    }
    else if (e.Key == Key.Back)
    {
        (DataContext as MainViewModel)?.OnKeyInput("Back");
    }
}
```

Einordnung:

- MVVM „rein“ würde hierfür eher Behavior/Attached Properties oder InputBindings verwenden.
- Im Projekt ist die Trennung trotzdem erkennbar: Der Code-Behind entscheidet nur *welche Taste*, das ViewModel entscheidet *was es bedeutet*.

Kapitel 3.3 : ViewModel – Bindbare Properties und Anwendungslogik

Das zentrale ViewModel ist **MainViewModel**.

- Es stellt **Properties** für die Anzeige bereit:
DisplayText, ExpressionText, IsRadianMode, AngleModeText
- Es stellt **Commands** für UI-Aktionen:
DigitCommand, OperationCommand, EqualsCommand, usw.
- Es stellt Orchestrierung:
es nutzt **CalculatorModel** und **CalculationService** zur Berechnung.

Grundlage für Binding ist **INotifyPropertyChanged**. Im Projekt wird das über **BaseViewModel** umgesetzt:

Code-Beispiel (BaseViewModel – PropertyChanged):

.../Taschenrechner/ViewModels/BaseViewModel.cs

```

public class BaseViewModel : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler PropertyChanged;

    protected virtual void OnPropertyChanged([CallerMemberName] string propertyName)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }

    protected bool SetProperty<T>(ref T storage, T value, [CallerMemberName] string propertyName)
    {
        if (Equals(storage, value))
            return false;

        storage = value;
        OnPropertyChanged(propertyName);
        return true;
    }
}

```

Einordnung:

- **SetProperty** ist das zentrale Muster:
Feld setzen → **PropertyChanged** auslösen → UI aktualisiert sich automatisch.

Das MainViewModel nutzt genau diese Mechanik:

Code-Beispiel (MainViewModel – bindbare Properties):

.../Taschenrechner/ViewModels/MainViewModel.cs

```

public string DisplayText
{
    get => _displayText;
    set => SetProperty(ref _displayText, value);
}

public string ExpressionText
{
    get => _expressionText;
    set => SetProperty(ref _expressionText, value);
}

public bool IsRadianMode
{
    get => _isRadianMode;
    set => SetProperty(ref _isRadianMode, value);
}

public string AngleModeText => IsRadianMode ? "RAD" : "DEG";

```

Wichtig:

AngleModeText ist eine abgeleitete Property. Sie wird nicht direkt gesetzt, sondern ergibt sich aus **IsRadianMode**. Deshalb wird im Toggle explizit *OnPropertyChanged(nameof(AngleModeText))* ausgelöst (Details siehe Kapitel Commands).

Kapitel 4 : Datenbindung (Binding) im Projekt

Binding verbindet die View mit dem ViewModel, ohne dass die View aktiv Daten „abholen“ muss. Im Projekt sind die wichtigsten Bindings:

- **ExpressionText** → obere Anzeigezeile
- **DisplayText** → Hauptanzeige
- **AngleModeText** → „RAD/DEG“-Label

Code-Beispiel (Binding Display):

.../Taschenrechner/Views/MainWindow.xaml

```
<TextBlock Text="{Binding ExpressionText}" ... />
<TextBlock Text="{Binding AngleModeText}" ... />
<TextBlock Text="{Binding DisplayText}" ... />
```

Erklärung:

- Sobald das ViewModel `DisplayText = ...` setzt und dadurch **PropertyChanged** auslöst, aktualisiert WPF die TextBlock-Anzeige.
 - Es gibt im Code keine direkte Referenz „`TextBlock.Text = ...`“ – die UI bleibt deklarativ.
-

Kapitel 5 : Verwendung von Commands im Projekt

Commands sind die MVVM-Alternative zu Click-Eventhandlern. Das Projekt verwendet ein klassisches **RelayCommand**-Pattern:

Code-Beispiel (RelayCommand):

.../Taschenrechner/ViewModels/RelayCommand.cs

```

public class RelayCommand : ICommand
{
    private readonly Action<object> _execute;
    private readonly Predicate<object> _canExecute;

    public RelayCommand(Action<object> execute, Predicate<object> canExecute)
    {
        _execute = execute ?? throw new ArgumentNullException(nameof(execute));
        _canExecute = canExecute;
    }

    public bool CanExecute(object parameter) => _canExecute == null || _canExecute(parameter);

    public void Execute(object parameter) => _execute(parameter);

    public event EventHandler CanExecuteChanged
    {
        add { CommandManager.RequerySuggested += value; }
        remove { CommandManager.RequerySuggested -= value; }
    }
}

```

Erklärung:

- **Execute** kapselt die auszuführende Aktion.
- Optional kann über **CanExecute** die Aktivierung eines Buttons gesteuert werden (im Projekt derzeit nicht genutzt, aber vorbereitet).

Im MainViewModel werden die Commands im Konstruktor initialisiert:

Code-Beispiel (Command-Initialisierung):

.../Taschenrechner/ViewModels/MainViewModel.cs

```

public MainViewModel()
{
    _calculatorModel = new CalculatorModel();
    _calculationService = new CalculationService();

    DigitCommand = new RelayCommand(OnDigitPressed);
    OperationCommand = new RelayCommand(OnOperationPressed);
    SpecialFunctionCommand = new RelayCommand(OnSpecialFunctionPressed);
    ClearCommand = new RelayCommand(param => Clear());
    ClearEntryCommand = new RelayCommand(param => ClearEntry());
    EqualsCommand = new RelayCommand(param => Calculate());
    BackspaceCommand = new RelayCommand(param => Backspace());
    MemoryStoreCommand = new RelayCommand(param => MemoryStore());
    MemoryRecallCommand = new RelayCommand(param => MemoryRecall());
    MemoryClearCommand = new RelayCommand(param => MemoryClear());
    ToggleAngleModeCommand = new RelayCommand(param => ToggleAngleMode());
}

```

Erklärung:

- UI-Buttons binden auf diese Commands.
- Das ViewModel ist damit der zentrale Ort für „User Intents“ (Ziffer drücken, Operator wählen, Ergebnis berechnen).

Ein besonders typisches MVVM-Detail ist die Aktualisierung einer abgeleiteten Property beim Umschalten von RAD/DEG:

Code-Beispiel (Toggle + PropertyChanged):

.../Taschenrechner/ViewModels/MainViewModel.cs

```

private void ToggleAngleMode()
{
    IsRadianMode = !IsRadianMode;
    OnPropertyChanged(nameof(AngleModeText));
}

```

Wichtig:

IsRadianMode löst ein PropertyChanged für sich selbst aus (über

SetProperty). AngleModeText ist aber abgeleitet – deshalb wird es zusätzlich explizit gemeldet.

Kapitel 6 : Kommunikation zwischen Komponenten

Im Projekt ist die Kommunikation bewusst direkt gehalten:

1. **View** → **ViewModel** über Commands (Button-Klicks)
2. **View** → **ViewModel** über delegierte Keyboard-Eingaben (OnKeyInput)
3. **ViewModel** → **Model/Service** über Methodenaufrufe
4. **ViewModel** → **View** indirekt über Binding (PropertyChanged)

Beispiel: „= **drücken**

- View ruft EqualsCommand auf:
Command="{Binding EqualsCommand}" (XAML)
- ViewModel führt Calculate() aus.
- CalculationService.EvaluateExpression(ExpressionText) berechnet das Ergebnis.
- ViewModel setzt DisplayText und ExpressionText.
- WPF aktualisiert die Anzeige automatisch.

Code-Beispiel (Calculate):

.../Taschenrechner/ViewModels/MainViewModel.cs

```

private void Calculate()
{
    if (_isError)
        return;

    try
    {
        double result = _calculationService.EvaluateExpression(Expression1
        DisplayText = result.ToString();
        ExpressionText += " = " + result;
        ExpressionText = result.ToString();
        _input = result.ToString();
    }
    catch (Exception ex)
    {
        DisplayText = "Fehler";
        ExpressionText = ex.Message;
        _isError = true;
    }
}

```

Erklärung:

- Die UI bekommt keine Exceptions zu sehen; sie bekommt nur neue Texte.
- Fehlerzustand wird intern über **_isError** verwaltet. Das ist praktisch, könnte aber später durch explizite Status-Properties (z. B. HasError, ErrorMessage) sauberer modelliert werden.

Kapitel 7 : Projektstruktur (Ordner, Dateien, Verantwortlichkeiten)

Die Struktur im Repository ist klar und entspricht der MVVM-Logik:

- **Taschenrechner/** (WPF-App)
 - **App.xaml / App.xaml.cs:** WPF-Startpunkt (StartupUri verweist auf MainWindow)

- **Views/**
 - `MainWindow.xaml`: UI + Binding + Command-Bindings
 - `MainWindow.xaml.cs`: KeyDown-Handling, Delegation an ViewModel
- **ViewModels/**
 - `BaseViewModel.cs`: **INotifyPropertyChanged**-Basis
 - `RelayCommand.cs`: ICommand-Implementierung
 - `MainViewModel.cs`: UI-Logik, Commands, Bindings
- **Models/**
 - `CalculatorModel.cs`: mathematische Grundfunktionen /Validierung
- **Services/**
 - `CalculationService.cs`: Auswertung von Ausdrücken
- **Helpers/**
 - `MathExpressionParser.cs`: demonstrativer Parser/Ersetzer
- **TaschenrechnerTests/**
 - `CalculatorModelTests.cs`: Unit-Tests für Model-Funktionen

Code-Beispiel (App StartupUri):

... /Taschenrechner/App.xaml

```
<Application ... StartupUri="/Views/MainWindow.xaml">
```

Einordnung:

- Das Projekt startet über die View; dort wird das ViewModel im XAML als DataContext erzeugt.
 - Tests sind aktuell auf das Model fokussiert – was sinnvoll ist, weil Model-Logik klar definiert und UI-unabhängig ist.
-

Kapitel 8 : Vorteile und mögliche Nachteile von MVVM im Kontext dieses Projekts

Kapitel 8.1 : Vorteile

- **Klare Trennung von UI und Logik:**
Rechenlogik liegt im **CalculatorModel** und **CalculationService**, nicht in der View.
- **Gute Erweiterbarkeit:**
Neue Buttons/Operationen werden meist durch neue Commands bzw. Erweiterungen in `OnSpecialFunctionPressed` ergänzt, ohne XAML grundlegend umzubauen.
- **Testbarkeit:**
Die Model-Tests zeigen, wie unabhängig vom UI getestet werden kann.

Code-Beispiel (Testbarkeit – Unit Test):

.../TaschenrechnerTests/CalculatorModelTests.cs

```
[Test]
public void Add_TwoNumbers_ReturnsCorrectSum()
{
    double result = _calculator.Add(5, 3);
    Assert.AreEqual(8, result);
}
```

Kapitel 8.2 : Mögliche Nachteile / Verbesserungsfelder

- **Direkte Instanziierung im XAML (kein DI):**
<viewModels:MainViewModel /> ist einfach, skaliert aber schlechter, wenn Services konfigurierbar oder austauschbar werden sollen.
- **Code-Behind für Keyboard-Handling:**
Funktional sauber, aber MVVM-puristisch wäre eine Lösung ohne Code-Behind (InputBindings/Behaviors).

- **Expression-Auswertung via DataTable.Compute:**

Für Demo/Übung ok, aber fachlich/technisch begrenzt (Parser-Robustheit, Funktionssupport, Sicherheits-/Validierungsfragen). Der vorhandene MathExpressionParser deutet bereits an, wie man perspektivisch sauberer werden könnte.

Kapitel 9 : Fazit

Das Projekt „Taschenrechner“ setzt MVVM in einer nachvollziehbaren, praxisnahen Form um: Die View beschreibt nur das UI und enthält keine Logik. (XAML + Binding + Commands), das ViewModel bündelt den UI-Workflow (Eingaben, Zustände, Fehlerbehandlung), und das Model kapselt die mathematische Fachlogik. Für den Projektumfang ist die Lösung sinnvoll balanciert: MVVM wird konsequent genutzt, ohne die Anwendung durch zusätzliche Infrastruktur (DI, Messenger-Frameworks, Behavior-Bibliotheken) unnötig aufzublähen.

Kapitel 10: Model im MVVM – CalculatorModel im Projekt „Taschenrechner“

Das **Model** bildet im MVVM-Pattern die fachliche Grundlage der Anwendung. Es enthält Berechnungen, Regeln und Validierungen – unabhängig davon, wie die Oberfläche aussieht oder wie der Benutzer interagiert. Im Projekt „Taschenrechner“ übernimmt diese Rolle die Klasse **CalculatorModel**. Wichtig ist dabei: Das Model kennt weder **XAML**, noch **Binding**, noch **Commands**. Es liefert ausschließlich fachliche Funktionen, die vom **ViewModel** aufgerufen werden.

1. Aufgaben und Abgrenzung des Models

Die Klasse **CalculatorModel** kapselt:

- grundlegende Rechenoperationen (**Add**, **Subtract**, **Multiply**, **Divide**)
- erweiterte Operationen (**Power**, **SquareRoot**)
- trigonometrische Funktionen (**Sin**, **Cos**, **Tan**)
- logarithmische Funktionen (**Log10**, **Ln**, **Log**)
- weitere Funktionen (**Factorial**, **Percent**)

Wichtige Begriffe:

- **Model**: fachliche Logik (ohne UI)
- **Validierung**: fachliche Prüfung von Eingaben
- **Exception**: standardisierte Fehlerkommunikation an aufrufende Schichten (hier: ViewModel/Service)

2. Konkretes Codebeispiel aus dem Projekt

Code-Beispiel (Model: **CalculatorModel.cs**)

.../Taschenrechner/Models/CalculatorModel.cs

```
public class CalculatorModel
{
    // Grundlegende Operationen
    public double Add(double a, double b) => a + b;
    public double Subtract(double a, double b) => a - b;
    public double Multiply(double a, double b) => a * b;

    public double Divide(double a, double b)
    {
        if (b == 0)
            throw new DivideByZeroException("Division durch Null ist nicht möglich");
        return a / b;
    }

    // Erweiterte Operationen
    public double Power(double baseNum, double exponent) => Math.Pow(baseNum, exponent);

    public double SquareRoot(double value)
    {
        if (value < 0)
            throw new ArgumentException("Keine reelle Wurzel für negative Zahlen");
        return Math.Sqrt(value);
    }

    // Trigonometrische Funktionen (im Bogenmaß)
    public double Sin(double angle) => Math.Sin(angle);
    public double Cos(double angle) => Math.Cos(angle);

    public double Tan(double angle)
    {
        if (Math.Cos(angle) == 0)
            throw new ArgumentException("Tangens ist an diesem Punkt nicht definiert");
        return Math.Tan(angle);
    }

    // Logarithmische Funktionen
    public double Log10(double value)
    {
        if (value <= 0)
            throw new ArgumentException("Logarithmus ist nur für positive Zahlen definiert");
        return Math.Log10(value);
    }
}
```

```

public double Ln(double value)
{
    if (value <= 0)
        throw new ArgumentException("Natürlicher Logarithmus ist nur für positive Werte definiert.");
    return Math.Log(value);
}

public double Log(double baseNum, double value)
{
    if (value <= 0 || baseNum <= 0 || baseNum == 1)
        throw new ArgumentException("Logarithmus-Parameter ungültig.");
    return Math.Log(value, baseNum);
}

// Weitere nützliche Funktionen
public double Factorial(double n)
{
    if (n < 0 || n != Math.Floor(n))
        throw new ArgumentException("Fakultät ist nur für nicht-negative Ganzzahlen definiert.");

    if (n == 0 || n == 1)
        return 1;

    double result = 1;
    for (int i = 2; i <= n; i++)
        result *= i;

    return result;
}

public double Percent(double value) => value / 100;
}

```

3. Erklärung des Codes im MVVM-Kontext

3.1 Grundoperationen als „reine“ Fachmethoden

Die Methoden **Add**, **Subtract** und **Multiply** sind bewusst als „One-Liner“ implementiert. Sie sind:

- deterministisch (gleiche Eingabe → gleiches Ergebnis)
- unabhängig von UI-Zustand
- leicht testbar

Das ist ein typisches Merkmal guter Model-Logik im MVVM-Kontext.

3.2 Validierung über Exceptions: Divide und SquareRoot

Die Methode **Divide** enthält eine fachliche Regel: Division durch Null ist nicht erlaubt.

- Bei $b == 0$ wird eine **DivideByZeroException** geworfen.
- Das Model trifft damit eine fachliche Entscheidung, nicht die Oberfläche.

Das ViewModel kann anschließend entscheiden, wie es reagiert (z. B. „Fehler“ anzeigen und einen internen Fehlerzustand setzen).

Im Projekt wird genau dieses Muster verwendet: Das ViewModel fängt Exceptions ab und setzt **DisplayText** auf „Fehler“.

Ähnlich arbeitet **SquareRoot**:

- Negative Werte werden durch eine **ArgumentException** abgewehrt.
- Dadurch ist die Regel „keine reelle Wurzel für negative Zahlen“ im Model verankert.

3.3 Trigonometrie: Tan mit Definitionsprüfung

Tan prüft vor der Berechnung:

- Wenn `Math.Cos(angle) == 0`, ist der Tangens an dieser Stelle nicht definiert.
- Das Model kommuniziert den fachlichen Fehler über eine **ArgumentException**.

Im Projekt ist zusätzlich wichtig: Der Kommentar „(im Bogenmaß)“ bedeutet, dass der Methodenparameter als **Radian** interpretiert wird. Die Umrechnung Grad $\leftrightarrow$ Radian ist deshalb sinnvollerweise im **ViewModel** angesiedelt (Benutzerinteraktion / Modusumschaltung), nicht im Model.

3.4 Fakultät: Fachliche Plausibilitätsprüfung

Factorial ist ein gutes Beispiel dafür, dass fachliche Logik nicht nur „Formeln“ sind:

- Fakultät ist nur für nicht-negative ganze Zahlen definiert.
- Das wird im Code exakt so geprüft:

- `n < 0` (negativ)
- `n != Math.Floor(n)` (nicht ganzzahlig)

Erst danach folgt die iterative Berechnung.

4. Warum dieses Model MVVM-konform ist

Die Klasse **CalculatorModel** ist im Sinne von MVVM sauber umgesetzt, weil sie:

- keine Abhängigkeit zu **WPF** besitzt (kein `INotifyPropertyChanged`, kein `ICommand`, kein `Window`, kein `TextBlock`)
- sich ausschließlich um **Domänenlogik** kümmert
- Fehler fachlich eindeutig über **Exceptions** kommuniziert
- sehr gut isoliert testbar ist (Unit-Tests sind im Projekt vorhanden)

5. Direkter Bezug zum restlichen Projekt (kurze Einordnung)

Im Projekt wird das Model im ViewModel verwendet, z. B. für Spezialfunktionen (`sin`, `cos`, `sqrt`, `n!`, ...). Das ViewModel übernimmt dabei die Aufgaben, die *nicht* ins Model gehören:

- Auswahl der Funktion anhand eines UI-Parameters (`Button/CommandParameter`)
 - Moduslogik (`RAD/DEG`) und ggf. Umrechnung
 - Fehlerbehandlung für die Anzeige (z. B. „Fehler“ im Display)
-

Kapitel 11 : Unit-Tests im MVVM-Projekt – CalculatorModelTests als Qualitätsnachweis für das Model

Die Tests in **TaschenrechnerTests/CalculatorModelTests.cs** zeigen sehr deutlich, wie MVVM die **Testbarkeit** verbessert:

Das **Model** (hier: CalculatorModel) ist vollständig unabhängig von der Oberfläche und kann deshalb ohne WPF-Runtime, ohne XAML und ohne ViewModel getestet werden.

Im MVVM-Kontext ist das ein zentraler Vorteil: Fachlogik wird in Klassen ausgelagert, die sich in Isolation prüfen lassen.

Die UI bleibt dadurch schlanker, und die Logik ist über Tests stabil abgesichert.

1. Einordnung – Was wird hier getestet und warum?

Die Datei **CalculatorModelTests.cs** testet ausschließlich Methoden aus **CalculatorModel**. Dabei decken die Tests drei Kategorien ab:

1. **Korrekte Ergebnisse** bei gültigen Eingaben (z. B. Add, Subtract, Multiply, Divide)
2. **Fehlerfälle** über Exceptions (Division durch 0, negative Quadratwurzel)
3. **Funktionen mit numerischer Toleranz** (Sin, Log10), weil Floating-Point-Berechnungen Rundungsabweichungen haben können

Wichtige Begriffe:

- **Unit-Test:** Testet eine einzelne Klasse/Methode isoliert.
- **Arrange – Act – Assert:** Standardstruktur zur Lesbarkeit von Tests.
- **Delta/Toleranz:** Vergleich von double-Werten mit Abweichung.

2. Konkretes Codebeispiel aus dem Projekt

Code-Beispiel (Unit-Tests: CalculatorModelTests.cs)

.../TaschenrechnerTests/CalculatorModelTests.cs

```
using Taschenrechner.Models;

namespace TaschenrechnerTests
{
    public class Tests
    {
        private CalculatorModel _calculator;

        [SetUp]
        public void Setup()
        {
            _calculator = new CalculatorModel();
        }

        [Test]
        public void Add_TwoNumbers_ReturnsCorrectSum()
        {
            // Arrange
            double a = 5;
            double b = 3;
            double expected = 8;

            // Act
            double result = _calculator.Add(a, b);

            // Assert
            Assert.AreEqual(expected, result);
        }

        [Test]
        public void Divide_ByZero_ThrowsException()
        {
            // Arrange
            double a = 6;
            double b = 0;

            // Act & Assert
            Assert.Throws<DivideByZeroException>(() => _calculator.Divide(a, b));
        }
    }
}
```

```

[Test]
public void Sin_ValidAngle_ReturnsCorrectValue()
{
    // Arrange
    double angleInRadians = Math.PI / 2; // 90 degrees
    double expected = 1.0;

    // Act
    double result = _calculator.Sin(angleInRadians);

    // Assert
    Assert.AreEqual(expected, result, 0.0000001);
}
}
}

```

3. Erklärung der Teststruktur (Arrange – Act – Assert)

Die Tests folgen durchgehend dem Schema:

- **Arrange:** Testdaten vorbereiten (Eingaben, erwartetes Ergebnis)
- **Act:** Methode ausführen
- **Assert:** Ergebnis prüfen

Dieses Muster hat zwei praktische Effekte:

1. Tests sind schneller zu lesen, weil jede Phase klar erkennbar ist.
2. Fehler lassen sich leichter lokalisieren (Fehler in Setup, in Ausführung oder in Erwartung).

4. Setup und Test-Isolation

Im Setup wird vor jedem Test ein neues CalculatorModel erzeugt:

Code-Beispiel (Setup)

.../TaschenrechnerTests/CalculatorModelTests.cs

```
private CalculatorModel _calculator;

[SetUp]
public void Setup()
{
    _calculator = new CalculatorModel();
}
```

Erklärung:

- Jeder Test startet mit einem frischen Zustand.
- Das reduziert Seiteneffekte und verhindert, dass Tests sich gegenseitig beeinflussen.
- Im MVVM-Kontext bedeutet das: Fachlogik bleibt reproduzierbar und zuverlässig prüfbar.

5. Tests für Exceptions – fachliche Regeln als definierte Erwartungen

Ein besonders wichtiger Test ist:

- **Divide_ByZero_ThrowsException**

Damit wird festgelegt,

dass `CalculatorModel.Divide(a,0)` **immer** eine **DivideByZeroException** werfen muss.

Code-Beispiel (Exception-Test)

.../TaschenrechnerTests/CalculatorModelTests.cs

```
[Test]
public void Divide_ByZero_ThrowsException()
{
    double a = 6;
    double b = 0;

    Assert.Throws<DivideByZeroException>(() => _calculator.Divide(a, b));
}
```

Erklärung im MVVM-Kontext:

- Das Model kommuniziert Fehler fachlich über Exceptions.
- Das ViewModel fängt diese später ab und entscheidet, wie die View reagiert (z. B. Anzeige „Fehler“).
- Der Test sichert dabei den „Vertrag“ des Models ab.

6. Floating-Point-Tests mit Delta (Sin, Log10)

Bei trigonometrischen/logarithmischen Funktionen werden double-Werte verglichen. Wegen Rundungsfehlern ist ein exakter Vergleich oft nicht stabil. Deshalb verwendet der Test ein **Delta**:

.../TaschenrechnerTests/CalculatorModelTests.cs

```
Assert.AreEqual(expected, result, 0.0000001);
```

Erklärung:

- Der Test akzeptiert kleine Abweichungen bis $1e-7$.
 - Das ist für mathematische Funktionen üblich und verhindert „flaky tests“.
-

Kurzes Zwischenfazit: Warum diese Tests für MVVM besonders wertvoll sind

Diese Tests zeigen, dass MVVM im Projekt nicht nur „Architektur auf Papier“ ist, sondern konkrete Vorteile bringt:

- **Fachlogik ist isoliert testbar**, weil sie im Model liegt.
- **Fehlerfälle sind reproduzierbar**, weil sie als Exceptions definiert sind.
- Die Tests bilden eine solide Basis, um die Anwendung später zu erweitern (neue Funktionen, neue Regeln), ohne alte Funktionalität zu gefährden.

Kapitel 12 : Testabdeckung erweitern und ViewModel-Tests einordnen (Projekt „Taschenrechner“)

Dieses Kapitel ergänzt die bestehende Testdatei **CalculatorModelTests.cs** um zwei Perspektiven:

- Erstens, welche fachlichen Randfälle im **CalculatorModel** aktuell noch nicht abgedeckt sind.
- Zweitens, welche Tests auf **ViewModel-Ebene** sinnvoll sind und wie man sie im MVVM-Kontext sauber abgrenzt.

1. Sinnvolle Ergänzungen für CalculatorModelTests (Model-Ebene)

Die vorhandenen Tests decken Grundrechenarten und einige Spezialfälle bereits ab. Für ein stabileres fachliches Netz fehlen vor allem Tests für:

- weitere **Exception-Verträge** (z. B. ungültige Wertebereiche)
- numerische Grenzfälle (z. B. Werte nahe an Definitionslücken)
- Methoden, die zwar implementiert sind, aber nicht geprüft werden (**Percent, Log, Ln, Tan, Cos**)

Wichtige Begriffe:

- **Randfall:** gültige, aber ungewöhnliche Eingaben (z. B. 0, sehr groß, sehr klein).
- **Fehlervertrag:** definierte Exception bei ungültiger Eingabe.

1.1 Weitere Exception-Tests für Logarithmen

Im Model ist fachlich festgelegt:

- `Log10(value)` wirft bei `value <= 0` eine **ArgumentException**
- `Ln(value)` wirft bei `value <= 0` eine **ArgumentException**

Code-Beispiel (Erweiterungsvorschlag – Log10/Ln Fehlerfälle)

.../TaschenrechnerTests/CalculatorModelTests.Logarithms.cs

```
using Taschenrechner.Models;

namespace TaschenrechnerTests
{
    public partial class Tests
    {
        [Test]
        public void Log10_Zero_ThrowsException()
        {
            Assert.Throws<ArgumentException>(() => _calculator.Log10(0));
        }

        [Test]
        public void Log10_Negative_ThrowsException()
        {
            Assert.Throws<ArgumentException>(() => _calculator.Log10(-1));
        }

        [Test]
        public void Ln_Zero_ThrowsException()
        {
            Assert.Throws<ArgumentException>(() => _calculator.Ln(0));
        }

        [Test]
        public void Ln_Negative_ThrowsException()
        {
            Assert.Throws<ArgumentException>(() => _calculator.Ln(-0.5));
        }
    }
}
```

Erklärung:

Diese Tests sichern ab, dass die fachliche Regel „Logarithmen nur für positive Zahlen“ langfristig erhalten bleibt.

1.2 Test für Log(baseNum, value) – ungültige Basis

Die Methode `Log(baseNum, value)` definiert drei ungültige Fälle:

- `value <= 0`
- `baseNum <= 0`

- `baseNum == 1`

Code-Beispiel (Erweiterungsvorschlag – Log Basis/Value Validierung)

.../TaschenrechnerTests/CalculatorModelTests.LogBase.cs

```
using Taschenrechner.Models;

namespace TaschenrechnerTests
{
    public partial class Tests
    {
        [Test]
        public void Log_BaseIsOne_ThrowsException()
        {
            Assert.Throws<ArgumentException>(() => _calculator.Log(1, 10))
        }

        [Test]
        public void Log_BaseIsZero_ThrowsException()
        {
            Assert.Throws<ArgumentException>(() => _calculator.Log(0, 10))
        }

        [Test]
        public void Log_ValueIsZero_ThrowsException()
        {
            Assert.Throws<ArgumentException>(() => _calculator.Log(10, 0))
        }
    }
}
```

1.3 Tan – Test der Definitionslücke

Im Model wird `Tan(angle)` abgewehrt, wenn `Math.Cos(angle) == 0`. Typische Stelle: `angle =  $\pi/2$` .

Code-Beispiel (Erweiterungsvorschlag – Tan nicht definiert)

.../TaschenrechnerTests/CalculatorModelTests.Tan.cs

```

using Taschenrechner.Models;

namespace TaschenrechnerTests
{
    public partial class Tests
    {
        [Test]
        public void Tan_PiOver2_ThrowsException()
        {
            Assert.Throws<ArgumentException>(() => _calculator.Tan(Math.PI
        }
    }
}

```

Hinweis:

Je nach Floating-Point-Rundung kann `Math.Cos(Math.PI / 2)` nicht exakt 0 sein. Da dein Model **exakt** `== 0` prüft, ist dieser Test zugleich ein Qualitätsindikator: Wenn der Test fehlschlägt, ist es kein „Testproblem“, sondern ein Hinweis darauf, dass die fachliche Prüfung in `Tan` robuster formuliert werden sollte (z. B. mit Toleranzbereich). Das ist ein typischer Nutzen von Unit-Tests: Sie machen implizite Annahmen sichtbar.

1.4 Factorial – ungültige Eingaben (nicht ganzzahlig, negativ)

Aktuell wird nur ein gültiger Fall getestet (5!). Sinnvoll sind zusätzlich:

- negativ (z. B. -1)
- nicht ganzzahlig (z. B. 2.5)

Code-Beispiel (Erweiterungsvorschlag – Factorial Validierung)

.../TaschenrechnerTests/CalculatorModelTests.Factorial.cs

```

using Taschenrechner.Models;

namespace TaschenrechnerTests
{
    public partial class Tests
    {
        [Test]
        public void Factorial_Negative_ThrowsException()
        {
            Assert.Throws<ArgumentException>(() => _calculator.Factorial(-
        }

        [Test]
        public void Factorial_NonInteger_ThrowsException()
        {
            Assert.Throws<ArgumentException>(() => _calculator.Factorial(2
        }
    }
}

```

1.5 Percent – einfache Ergebnisprüfung

Percent(value) ist fachlich simpel, sollte aber trotzdem abgesichert sein:

Code-Beispiel (Erweiterungsvorschlag – Percent)

.../TaschenrechnerTests/CalculatorModelTests.Percent.cs

```

using Taschenrechner.Models;

namespace TaschenrechnerTests
{
    public partial class Tests
    {
        [Test]
        public void Percent_50_Returns0Point5()
        {
            double result = _calculator.Percent(50);
            Assert.AreEqual(0.5, result, 0.0000001);
        }
    }
}

```

2. ViewModel-Tests im MVVM-Kontext – was sinnvoll ist (und was nicht)

In MVVM sind ViewModels grundsätzlich testbar, aber nicht alles lohnt sich gleich stark. Für dein Projekt sind ViewModel-Tests insbesondere dort sinnvoll, wo:

- Zustandslogik umgesetzt ist (z. B. `_isError`, Reset-Verhalten)
- Strings aufgebaut werden (z. B. `ExpressionText`)
- Commands die zentrale Steuerlogik darstellen

Weniger sinnvoll (oder deutlich aufwendiger) sind Tests, die WPF-Details prüfen:

- Layout (Grid-Reihen/Spalten)
- Styles, Farben, Templates
- konkrete Button-Positionen

Diese Themen gehören in UI-Tests oder werden visuell geprüft.

2.1 Typische ViewModel-Testfälle aus MainViewModel

Aus `MainViewModel` ergeben sich naheliegende Testszenarien:

1. **DigitCommand** aktualisiert `DisplayText` und `ExpressionText` korrekt
2. **OperationCommand** fügt Operator mit Leerzeichen ein und setzt `DisplayText` zurück
3. **EqualsCommand** setzt nach erfolgreicher Berechnung `DisplayText` auf Ergebnis und setzt Expression zurück
4. **ToggleAngleModeCommand** toggelt `IsRadianMode` und beeinflusst `AngleModeText`

Ein Beispiel (konzeptionell) für einen reinen Logiktest:

Code-Beispiel (ViewModel-Test – Konzept, ohne WPF)

.../TaschenrechnerTests/MainViewModelTests.Concept.cs

```

using Taschenrechner.ViewModels;

namespace TaschenrechnerTests
{
    public class MainViewModelTests
    {
        [Test]
        public void ToggleAngleMode_Toggles_IsRadianMode()
        {
            var vm = new MainViewModel();
            bool initial = vm.IsRadianMode;

            vm.ToggleAngleModeCommand.Execute(null);

            Assert.AreNotEqual(initial, vm.IsRadianMode);
        }
    }
}

```

Erklärung:

- Das testet keine UI, sondern reinen ViewModel-Zustand.
- Der Test ist genau deshalb möglich, weil MVVM UI und Logik trennt.

2.2 Praktischer Hinweis – Testbarkeit des ViewModels erhöhen

Aktuell erzeugt MainViewModel seine Abhängigkeiten selbst:

- `new CalculatorModel()`
- `new CalculationService()`

Das ist für kleine Projekte in Ordnung, macht aber ViewModel-Tests unflexibler (z. B. wenn man Fehlerfälle gezielt simulieren will). Für bessere Testbarkeit würde man typischerweise:

- Konstruktor-Injection verwenden (Interfaces, z. B. `ICalculationService`)
- im Test Fake/Mock bereitstellen

Das ist keine Pflicht für das Projekt, aber der nächste logische Schritt, wenn die Anwendung wächst.

Anhang : Taschenrechner herunterladen und mit Visual Studio Code starten (Windows – einfach erklärt)

In diesem Anhang lernst du Schritt für Schritt, wie du das Projekt „**Taschenrechner**“ von GitHub auf deinen Windows-PC lädst, wie du **Visual Studio Code** (kostenlos) installierst und wie du das Programm startest.

1. Was du brauchst (Windows)

Du brauchst drei Dinge:

1. **Visual Studio Code** (zum Öffnen des Codes)
2. **.NET 8 SDK** (damit Windows das Programm bauen und starten kann)
3. Optional: **Git** (zum „Klonen“ – das ist die beste Download-Methode, aber nicht zwingend)

Voraussetzungen (Windows)

1) Visual Studio Code	Editor (kostenlos)
2) .NET 8 SDK	Zum Bauen & Starten
3) Git (optional)	Für „git clone“

Hinweis: WPF läuft nur unter Windows.

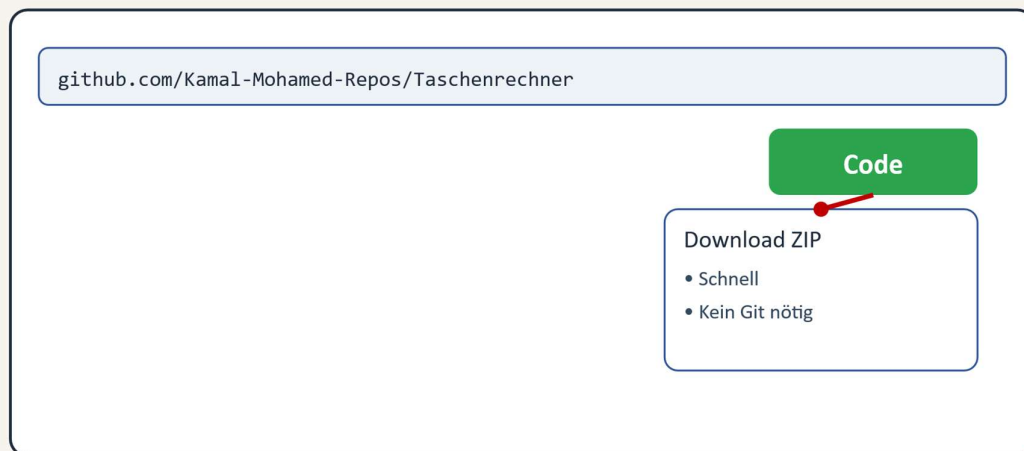
2. Projekt herunterladen

Du hast zwei einfache Möglichkeiten. Wenn du Anfänger bist, ist **ZIP** am leichtesten.

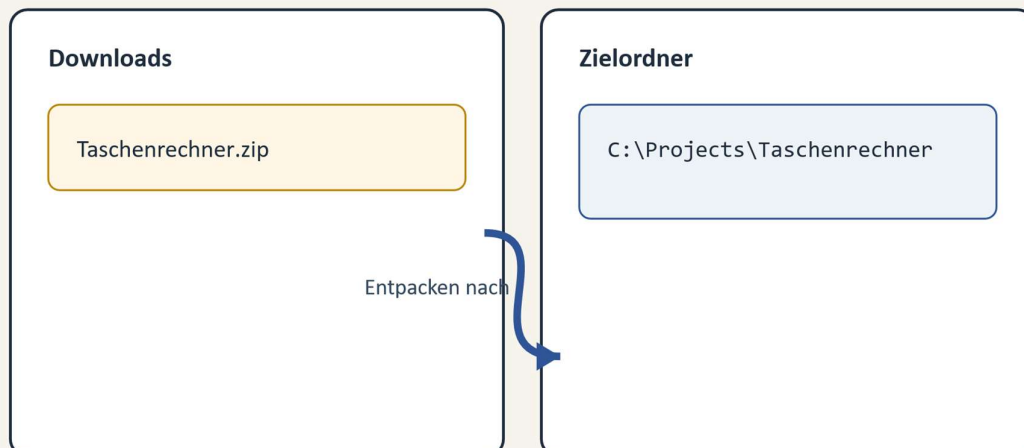
2.1 Variante A – Download als ZIP (am einfachsten)

1. Öffne GitHub im Browser und gehe zum Repository:
Kamal-Mohamed-Repos/Taschenrechner
2. Klicke auf den grünen Button **Code**.
3. Klicke auf **Download ZIP**.
4. Entpacke die ZIP-Datei (z. B. nach C:\Projects\Taschenrechner).

Repository als ZIP herunterladen



ZIP entpacken



2.2 Variante B – Mit Git klonen (empfohlen, wenn du Git installiert hast)

1. Öffne **PowerShell** (Windows-Suche → „PowerShell“).
2. Wechsle in deinen Projektordner:

```
cd C:\Projects
```

3. Kclone das Repository:

```
git clone https://github.com/Kamal-Mohamed-Repos/Taschenrechner.git
```

Alternative: Mit Git klonen

```
PS C:\Projects> git clone https://github.com/Kamal-Mohamed-Repos/Taschenrechner.git
Cloning into 'Taschenrechner'...
done.
```

Tipp: Git ist optional. ZIP reicht für den Einstieg.

3. Visual Studio Code (kostenlos) installieren

1. Öffne im Browser die Seite „Visual Studio Code Download“ (Microsoft).
2. Lade die Windows-Version herunter.
3. Starte die Installation und klicke dich mit **Weiter** durch.
4. Praktisch: Haken setzen bei
 - „Add to PATH“
 - „Open with Code“ (Rechtsklick-Menü)

Visual Studio Code herunterladen (kostenlos)

Download für Windows

- VS Code (User Installer)
- Kostenlos
- Für Windows 10/11

Download

Quelle: Microsoft (offizielle Seite)

VS Code installieren: empfohlene Optionen

Setup-Optionen

☒ Add to PATH (empfohlen)

☒ Open with Code (Rechtsklick-Menü)

Weiter → Installieren

4. .NET 8 SDK installieren (damit das Programm starten kann)

1. Öffne die Seite „.NET 8 SDK Download“ (Microsoft).
2. Installiere das **.NET 8 SDK**.
3. Prüfe die Installation:
 - Öffne PowerShell
 - Tippe:

```
dotnet --version
```

Wenn eine Version wie 8.x.x angezeigt wird, ist alles korrekt.

.NET 8 SDK herunterladen (für Start/Build)

.NET SDK 8.x (Windows)

- Installiert „dotnet“
- Erforderlich für WPF (.NET 8)
- Kostenlos

Download

Quelle: Microsoft (offizielle .NET Seite)

Installation prüfen

```
PS C:\> dotnet --version
```

```
8.0.x
```

Wenn hier eine 8.x-Version steht, ist das SDK korrekt installiert.

5. Projekt in VS Code öffnen

1. Öffne den Ordner, in den du das Projekt entpackt/geklopnt hast.
2. Rechtsklick in den Ordner → **Open with Code** (wenn aktiviert).
Alternativ: VS Code öffnen → **File** → **Open Folder** → Projektordner auswählen.

Woran erkennst du den richtigen Ordner?

Im Hauptordner liegt die Datei: **Taschenrechner.sln**

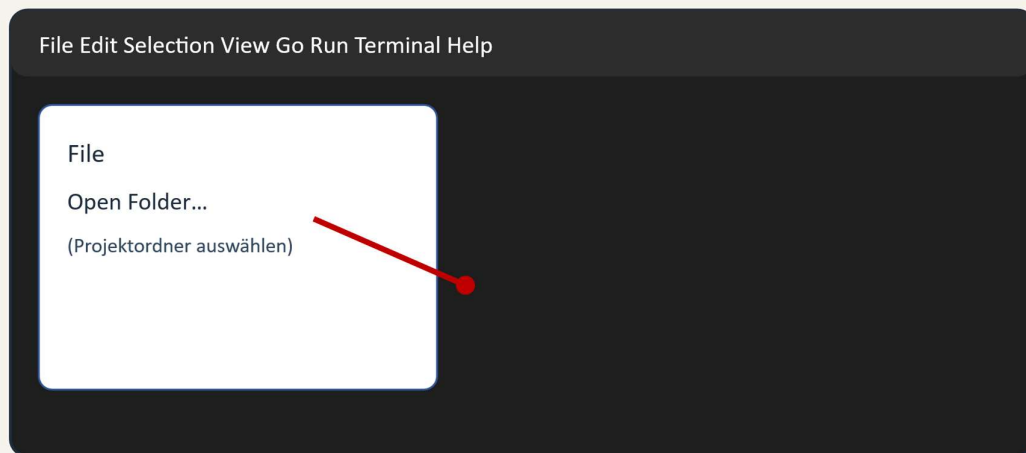
Richtigen Ordner erkennen

Der Projektordner enthält:

Taschenrechner.sln

Tipp: Öffne in VS Code am besten den Ordner, in dem die .sln liegt.

Projekt in VS Code öffnen



6. Programm starten (Windows)

Jetzt wird das Projekt gebaut und gestartet.

1. In VS Code oben: **Terminal** → **New Terminal**
2. Im Terminal gibst du folgenden Befehl ein:

```
dotnet run --project Taschenrechner\Taschenrechner.csproj
```

Wenn alles richtig ist, startet das Fenster „**Rechner**“.

Programm starten

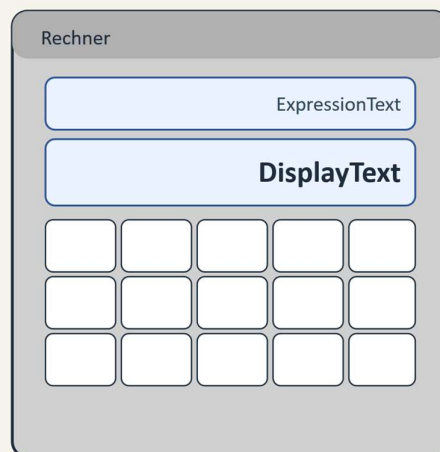
```
PS C:\Projects\Taschenrechner> dotnet run --project Taschenrechner\Taschenrech
```

Build succeeded.

Starting...

Danach öffnet sich das Fenster „Rechner“.

Ergebnis: Anwendung läuft



Oberfläche aus MainWindow.xaml, Logik im MainViewModel.

7. Häufige Fehler (kurz) und was du dann machst

- **Fehler:** dotnet wird nicht erkannt
Lösung: .NET 8 SDK installieren, danach PC neu starten.
- **Fehler:** Repository kann nicht geladen werden (privat/kein Zugriff)
Lösung: Mit dem richtigen GitHub-Account anmelden und Zugriff prüfen.